

1. Access Modifier kya hota hai?

Access modifier decide karta hai ki class ke member ko kahan se access kar sakte hain.

Modifier	Same Class	Child/Derived Class	Object se	
public	✓	✓	✓	
protected	✓	✓	✗	
private	✓	✗ Directly	✗	

```
</> C++  
  
#include <iostream>  
using namespace std;  
  
class A  
{  
public:  
    int A_public;  
  
protected:  
    int A_protected;  
  
private:  
    int A_private;  
  
public:  
  
    void setValues()  
    {  
        A_private = 10;    // ✓ allowed  
        A_protected = 20;  // ✓ allowed  
        A_public = 30;     // ✓ allowed  
    }  
};
```

```
class B : public A
{
public:

    int B_public;

protected:
    int B_protected;

private:
    int B_private;

public:

    void setValues()
    {
        // A_private = 10;           // ✗ private member of A
        A_protected = 20;           // ✓ protected member of A
        A_public = 30;             // ✓ public member of A
    }
}
```

```
    B_private = 50;        // ✓ own private member
    B_protected = 60;      // ✓ own protected member
    B_public = 40;         // ✓ own public member
}

void show()
{
    cout << "This is child class B" << endl;

    cout << "A_protected = "
         << A_protected << endl;

    cout << "A_public = "
         << A_public << endl;

    cout << "B_public = "
         << B_public << endl;

    cout << "B_protected = "
         << B_protected << endl;

    cout << "B_private = "
         << B_private << endl;
}
```



```
int main()
{
    cout << "Hello Tarun" << endl;

    B obj;

    obj.A_public = 30;          // ✓ allowed

    // obj.A_protected = 20; // ✗ not accessible
    // obj.A_private = 10;   // ✗ not accessible

    obj.B_public = 40;          // ✓ allowed

    // obj.B_protected = 60; // ✗ not accessible
    // obj.B_private = 50;   // ✗ not accessible

    obj.setValues();
    obj.show();

    return 0;
}
```



7. Sabse Important — public inheritance

</> C++

```
class B : public A
```

Iska meaning:

A ke members B me kaise behave karenge?

A ka member	B ke andar	B obj se
A_private	✗ inaccessible	✗
A_protected	✓ protected	✗
A_public	✓ public	✓

```
class B : protected A
```

Base class ke:

```
public    → protected
protected → protected
private  → inaccessible
```



Matlab B ke object se A ke public/protected members access nahi honge, but B ke andar access ho sakte hain. PDF me bhi यही protected inheritance दिखाया गया है. deleted

A ka member	B ke andar	B obj se
A_public	protected	✗
A_protected	protected	✗
A_private	✗ inaccessible	✗

```
class B : private A
```

Base class ke:

```
public    → private  
protected → private  
private  → inaccessible
```



Matlab B ke object se A ka kuch bhi directly access nahi hoga, lekin B ke andar A ke **public/protected** members access ho sakte hain.

```
B : public A  
public    → public  
protected → protected
```

```
B : protected A  
public    → protected  
protected → protected
```

```
B : private A  
public    → private  
protected → private
```

Base ka **private** member teeno inheritance me directly accessible nahi hota.

```
class B : public A
{
public:
    void show()
    {
        cout << A_public << endl;    // ✓
        cout << A_protected << endl;  // ✓
        // cout << A_private << endl; // ✗
    }
};
```

// 2. PROTECTED INHERITANCE

```
class C : protected A
{
public:
    void show()
    {
        cout << A_public << endl;    // ✓
        cout << A_protected << endl;  // ✓
        // cout << A_private << endl; // 
    }
};
```



```
</> C++
```

```
// 3. PRIVATE INHERITANCE
```

```
class D : private A
```

```
{
```

```
public:
```

```
    void show()
```

```
    {
```

```
        cout << A_public << endl;    // ✓
```

```
        cout << A_protected << endl; // ✓
```

```
        // cout << A_private << endl; // ✗
```

```
    }
```

```
};
```

```
</> C++
```

```
#include <iostream>
```

```
using namespace std;
```

```
class A
```

```
{
```

```
private:
```

```
    void A_private()
```

```
    {
```

```
        cout << "class A ka private member function" << endl;
```

```
    }
```

```
// ✗ PRIVATE FUNCTION
```

```
// sirf class A ke andar call ho sakta hai
```

```
// child class B bhi call nahi kar sakti
```

```
// object bhi call nahi kar sakta
```

```
protected:
```

protected:

```
void A_protected()
{
    cout << "class A ka protected member function" << endl;
}

//  PROTECTED FUNCTION
// class A ke andar call ho sakta hai
// child class B ke andar call ho sakta hai
//  object se call nahi hota
```

public:

```
void A_public()
{
    cout << "class A ka public member function" << endl;
}

//  PUBLIC FUNCTION
// class A ke andar call
// child class B ke andar call
// object se bhi call
};
```

```
// PRIVATE INHERITANCE
```

```
class B : private A
```

```
{
```

```
private:
```

```
    void B_private()
```

```
    {
```

```
        cout << "child class B ka private member function" << endl;
```

```
    }
```

```
// ❌ object se call nahi
```

```
// ✅ sirf class B ke andar
```

```
protected:
```

```
    void B_protected()
```

```
    {
```

```
        cout << "child class B ka protected member function" << endl;
```

```
    }
```

```
// ❌ object se call nahi
```

```
// ✅ class B ke andar
```

```
public:

void B_public()
{
    cout << "child class B ka public member function" << endl;

    // A_private();
    // ❌ private member of A
    // inheritance ke baad bhi access nahi

    A_protected();
    // ✅ allowed
    // A ka protected → B me PRIVATE bana
    // lekin class B ke andar access allowed

    A_public();
    // ✅ allowed
    // A ka public → private inheritance ke baad
    // B me PRIVATE bana
    // lekin class B ke andar access allowed
```



```
        B_private();  
        //  allowed  
        // same class B  
  
        B_protected();  
        //  allowed  
        // same class B  
    }  
  
    // WRAPPER FUNCTION  
  
    void accessA_from_object()  
    {  
        A_public();  
        //  indirect access  
  
        A_protected();  
        //  indirect access  
    }  
};
```



```
int main()
{
    cout << "Hello Tarun" << endl;

    B obj;

    // obj.A_public();
    // ✗ object se access nahi
    // private inheritance ke baad
    // A ka public → PRIVATE ban gaya

    // obj.A_protected();
    // ✗ object se access nahi
    // protected function

    // obj.B_private();
    // ✗ private member of B

    // obj.B_protected();
    // ✗ protected member of B
```

```

    obj.B_public();
    // ✅ allowed
    // B_public() public hai

    obj.accessA_from_object();
    // ✅ allowed
    // wrapper function ke through
    // A_public() aur A_protected() indirectly call
    // ho rahe hain

    return 0;
}

```

```
class B : private A
```

ka matlab:

A ka function	B ke andar
A_private()	❌ Access nahi
A_protected()	🔒 Private
A_public()	🔒 Private

PDF me:

```
</> C++  
  
void accessA_from_object()  
{  
    A_public();  
    A_protected();  
}
```

Iska purpose hai **object** ko **directly** A ke functions access nahi karne dena, balki B ke **public function** ke through **indirectly** access karwana.  delete2

PRIVATE

→ Only same class

PROTECTED

→ Same class + Child class

PUBLIC

→ Same class + Child class + Object

private inheritance:

A public → B private

A protected → B private

A private → inaccessible

```
class B : protected A
{
private:
    void B_private()
    {
        cout << "B private" << endl;
    }

protected:
    void B_protected()
    {
        cout << "B protected" << endl;
    }

public:
    void B_public()
    {
        cout << "B public" << endl;

        // A_private();    // ❌ A ka private → access nahi

        A_protected();    // ✅ allowed
        A_public();        // ✅ allowed
    }
}
```



```
public:
    void B_public()
    {
        cout << "B public" << endl;

        // A_private();    // ❌ A ka private → access nahi

        A_protected();    // ✅ allowed
        A_public();        // ✅ allowed

        B_private();      // ✅ allowed
        B_protected();     // ✅ allowed
    }

    // Wrapper function
    void accessA_from_object()
    {
        A_public();        // ✅ indirect access
        A_protected();     // ✅ indirect access
    }
};
```



```

class B : public A
{
public:

    void B_public()
    {
        // A_private();    // ❌ Access nahi

        A_protected();    // ✅ allowed
        A_public();        // ✅ allowed
    }
};

```

```

class A
{
private:

    int A_private;

    // PRIVATE CONSTRUCTOR
    A(int x)
    {
        A_private = x;
        cout << "[class A ka PRIVATE] constructor call hua"
              << endl;
    }
}

```

protected:

```
float A_protected;
```

```
// PROTECTED CONSTRUCTOR
```

```
A(float x)
```

```
{
```

```
    A_protected = x;
```

```
    cout << "[class A ka PROTECTED] constructor call hua"  
         << endl;
```

```
}
```

public:

```
string A_public;
```

```
// PUBLIC CONSTRUCTOR
```

```
A(string x)
```

```
{
```

```
    A_public = x;
```

```
    cout << "[class A ka PUBLIC] constructor call hua"  
         << endl;
```



```
// PUBLIC INHERITANCE
class B : public A
{
private:

    int B_private;

// PRIVATE CONSTRUCTOR
    B(int x) : A("from B private")
    {
        B_private = x;
        cout << "[class B ka PRIVATE] constructor call hua"
              << endl;
    }
}
```

protected:

```
float B_protected;
```

```
// PROTECTED CONSTRUCTOR
```

```
B(float x) : A(x)
```

```
{
```

```
    B_protected = x;
```

```
    cout << "[class B ka PROTECTED] constructor call hua"
```

```
    << endl;
```

```
}
```

public:

```
string B_public;
```

```
// PUBLIC CONSTRUCTOR
```

```
B(string x) : A(x)
```

```
{
```

```
    B_public = x;
```

```
    cout << "[class B ka PUBLIC] constructor call hua"
```

```
    << endl;
```



```
// STATIC WRAPPER  
// Private constructor ko indirectly call karne ke liye  
  
static B createWithInt(int x)  
{  
    cout << "STATIC createWithInt() called" << endl;  
  
    return B(x);  
}  
  
// Protected constructor ko indirectly call karne ke liye  
  
static B createWithFloat(float x)  
{  
    cout << "STATIC createWithFloat() called" << endl;  
  
    return B(x);  
}  
};
```

```
int main()
{
    // CASE 1: PUBLIC CONSTRUCTOR

    B b1("Tarun");

    // ✅ Object se direct allowed
    // B(string) public hai

    // CASE 2: PRIVATE CONSTRUCTOR

    // B b2(10);
    // ❌ Not allowed
    // B(int) private hai

    B b2 = B::createWithInt(10);
    // ✅ Wrapper ke through indirect access

    // CASE 3: PROTECTED CONSTRUCTOR
```



```

    // B b3(3.14f);
    // ❌ Not allowed
    // B(float) protected hai

    B b3 = B::createWithFloat(3.14f);
    // ✅ Wrapper ke through indirect access

    return 0;
}

```

```

class B : protected A
{
private:
    int B_private;

    // PRIVATE CONSTRUCTOR OF B
    B(int x) : A("from B private")
    {
        B_private = x;

        cout << "[class B ka PRIVATE] constructor call hua"
              << endl;
    }
}

```

```
protected:
    float B_protected;

    // PROTECTED CONSTRUCTOR OF B
    B(float x) : A(x)
    {
        B_protected = x;

        cout << "[class B ka PROTECTED] constructor call hua"
              << endl;
    }
```

```
public:
    string B_public;

    // PUBLIC CONSTRUCTOR OF B
    B(string x) : A(x)
    {
        B_public = x;

        cout << "[class B ka PUBLIC] constructor call hua"
              << endl;
    }

    // STATIC WRAPPER
    // B ke private constructor ko indirectly call karne ke liye

    static B createWithInt(int x)
    {
        cout << "STATIC createWithInt() called" << endl;

        return B(x);
    }
```



```
// STATIC WRAPPER
// B ke protected constructor ko indirectly call karne ke liye

static B createWithFloat(float x)
{
    cout << "STATIC createWithFloat() called" << endl;

    return B(x);
}

};

int main()
{
    // CASE 1: PUBLIC CONSTRUCTOR

    B b1("Tarun");

    // ✅ Allowed
    // B(string) public hai
```

```
// CASE 2: PRIVATE CONSTRUCTOR

// B b2(10);
// ❌ Not allowed
// B(int) private hai

B b2 = B::createWithInt(10);

// ✅ Allowed
// static wrapper ke through
// private constructor indirectly call hua

// CASE 3: PROTECTED CONSTRUCTOR

// B b3(3.14f);
// ❌ Not allowed
// B(float) protected hai

B b3 = B::createWithFloat(3.14f);
```

```

    B b3 = B::createWithFloat(3.14f);

    //  Allowed
    // static wrapper ke through
    // protected constructor indirectly call hua

    return 0;
}

```

```

// PRIVATE INHERITANCE
class B : private A
{
private:
    int B_private;

    // PRIVATE CONSTRUCTOR OF B
    B(int x) : A("from B private")
    {
        B_private = x;

        cout << "[class B ka PRIVATE] constructor call hua" << endl;
    }
}

```

```
protected:
    float B_protected;

    // PROTECTED CONSTRUCTOR OF B
    B(float x) : A(x)
    {
        B_protected = x;

        cout << "[class B ka PROTECTED] constructor call hua" << endl;
    }

public:
    string B_public;

    // PUBLIC CONSTRUCTOR OF B
    B(string x) : A(x)
    {
        B_public = x;

        cout << "[class B ka PUBLIC] constructor call hua" << endl;
    }
```



```
// STATIC WRAPPER
static B createWithInt(int x)
{
    cout << "STATIC createWithInt() called" << endl;

    return B(x);
}

// STATIC WRAPPER
static B createWithFloat(float x)
{
    cout << "STATIC createWithFloat() called" << endl;

    return B(x);
}
};
```

```
int main()
{
    // 1. PUBLIC CONSTRUCTOR

    B b1("Tarun");

    // ✅ Allowed
    // B(string) public hai

    // 2. PRIVATE CONSTRUCTOR

    // B b2(10);
    // ❌ Not allowed

    B b2 = B::createWithInt(10);

    // ✅ Wrapper ke through indirect access
```

```
// 3. PROTECTED CONSTRUCTOR  
  
// B b3(3.14f);  
// ❌ Not allowed  
  
B b3 = B::createWithFloat(3.14f);  
  
// ✅ Wrapper ke through indirect access  
  
return 0;  
}
```

```
class NodeB;  
class NodeC;  
  
class NodeA  
{  
public:  
  
    int dataA;  
  
    NodeA* nextA;  
  
    NodeB* nextB;  
  
    NodeC* nextC;  
};
```

</> C++

```
NodeA* nextA;
```

ka matlab:

NodeA ke paas doosre NodeA ko point karne wala pointer hai.

Example:

```
NodeA → NodeA → NodeA → NULL
```

2. Direct object kyu nahi?

Agar hum likh dein:

```
</> C++  
  
class NodeA  
{  
    NodeA nextA;    // ✖  
};
```

Problem:

```
NodeA  
  ↓  
NodeA  
  ↓  
NodeA  
  ↓  
NodeA  
  ↓  
...
```



Har `NodeA` ke andar ek aur `NodeA` chahive.

Har `NodeA` ke andar ek aur `NodeA` chahiye.

Isliye object ka size theoretically **infinite** ho jayega.

Compiler iska size calculate nahi kar sakta.

Isliye:

```
</> C++
```

```
NodeA nextA;    // ❌
```

ke badle:

```
</> C++
```

```
NodeA* nextA;    // ✅
```

use karte hain.

Pointer ka size fixed hota hai.

3. NodeB — NodeA ka Child

PDF me:

```
</> C++  
  
class NodeB : public NodeA  
{  
public:  
  
    int dataB;  
  
    NodeB* nextB;  
  
    NodeA* nextA;  
    NodeC* nextC;  
};
```

```
</> C++
```

```
NodeB* nextB;
```

hai.

Iska purpose:

```
NodeB → NodeB → NodeB
```

Same-level linked list.

Aur:

```
</> C++
```

```
NodeA* nextA;
```

ka purpose:

ka purpose:

NodeB



NodeA

Parent ko connect karna.

⟨/⟩ C++

```
class NodeC : public NodeB
{
public:

    int dataC;

    NodeC* nextC;

    NodeB* nextB;

    NodeA* nextA;
};
```

Ab hierarchy:

NodeA
↑
NodeB
↑
NodeC

NodeC ke paas:

⌞ C++

```
NodeC* nextC;
```

same-level nodes ke liye:

NodeC → NodeC → NodeC

NodeB* nextB :

NodeC



NodeB

parent ko connect karta hai.

NodeA* nextA :

NodeC



NodeB



NodeA

grand-parent ko connect kar sakta hai.

```
#include <iostream>
using namespace std;

// Forward declaration
class NodeB;
class NodeC;

// ===== NODE A =====

class NodeA
{
public:

    int dataA;

    // Same-level NodeA
    NodeA* nextA;

    // Child NodeB
    NodeB* nextB;
```

```
// Child NodeB
NodeB* nextB;

// Deep child NodeC
NodeC* nextC;

NodeA(int a = 0)
{
    dataA = a;

    nextA = nullptr;
    nextB = nullptr;
    nextC = nullptr;
}
};
```

```
class NodeB : public NodeA
{
public:

    int dataB;

    // Same-Level NodeB
    NodeB* nextB;

    // Parent NodeA
    NodeA* nextA;

    // Child NodeC
    NodeC* nextC;

    NodeB(int a = 0, int b = 0)
        : NodeA(a)
    {
        dataB = b;

        nextA = nullptr;
        nextB = nullptr;
    }
};
```

```
</> C++

    nextA = nullptr;
    nextB = nullptr;
    nextC = nullptr;
}
};

// ===== NODE C =====

class NodeC : public NodeB
{
public:

    int dataC;

    // Same-Level NodeC
    NodeC* nextC;

    // Parent NodeB
    NodeB* nextB;

    // Grand-parent NodeA
    NodeA* nextA;
```

```
// C++  
  
// Grand-parent NodeA  
NodeA* nextA;  
  
NodeC(int a = 0, int b = 0, int c = 0)  
    : NodeB(a, b)  
{  
    dataC = c;  
  
    nextA = nullptr;  
    nextB = nullptr;  
    nextC = nullptr;  
}  
};
```

```

int main()
{
    // Object
    NodeC obj_C(10, 20, 30);

    cout << "Object Values" << endl;

    cout << obj_C.dataA << endl;
    cout << obj_C.dataB << endl;
    cout << obj_C.dataC << endl;

    return 0;
}

```

Aapke PDF ka final table basically ye batata hai:

 delete4

Member	Type	Purpose
nextA	NodeA*	Same-level NodeA
nextB	NodeB*	Parent/child NodeB connection
nextC	NodeC*	Same/deep NodeC connection
objC	Direct object	Actual object/data

Nested Class kya hoti hai?

Jab ek class ke andar doosri class define karte hain, use **Nested Class** kehte hain.

```
</> C++  
  
class A  
{  
public:  
  
    int x = 10;  
  
    // Nested Class  
    class B  
    {  
    public:  
        int y = 20;  
  
        void show()  
        {  
            cout << "B class" << endl;  
        }  
    };  
};
```



a-46f9f20393c6

Yahan:

```
A = Outer class  
B = Nested/Inner class
```

Structure:

```
A  
└─ B
```

Object kaise banega?

Nested class ka object:

```
</> C++  
  
int main()  
{  
    A::B obj;  
  
    obj.show();  
}
```

A::B kyu?

Kyuki **B**, **A** ke andar defined hai.

```
</> C++
```

```
A::B obj;
```

ka meaning:

Class A ke andar wali class B ka object banao.

```
class Student
{
public:

    string name = "Tarun";

    // Nested Class
    class Address
    {
    public:

        string city = "Indore";

        void showAddress()
        {
            cout << "City: " << city << endl;
        }
    };
};
```

```

int main()
{
    Student::Address obj;

    obj.showAddress();

    return 0;
}

```

Output:

City: Indore

Concept	Meaning	Example
Nested Class	Class ke andar class	<code>class A { class B {}; };</code>
Nested Object	Class ke andar doosri class ka object/member	<code>class A { B obj; };</code>
Inheritance	Ek class doosri class se inherit kare	<code>class B : public A</code>